



SI100B: Introduction to Information Science and Technology



Lecture 9



EXCEPTIONS

EXAMPLE

```
a = int(input("Tell me one number:"))
b = int(input("Tell me another number:"))
print(a/b)
```

- ▶ User might input a character :(
- ▶ User might make b be 0 :(
- ▶ Use try/except around the problematic code

try:

```
a = int(input("Tell me one number:"))
b = int(input("Tell me another number:"))
print(a/b)
```

except:

```
print("Bug in user input.")
```



HANDLING SPECIFIC EXCEPTIONS

- ▶ Have **separate except clauses** to deal with a particular type of exception

```
try:
    a = int(input("Tell me one number: "))
    b = int(input("Tell me another number: "))
    print("a/b = ", a/b)
    print("a+b = ", a+b)
except ValueError:
    print("Could not convert to a number.")
except ZeroDivisionError:
    print("Can't divide by zero")
    print("a/b = infinity")
    print("a+b =", a+b)
except:
    print("Something went very wrong.")
```

*only execute
if these errors
come up*

*for all other
errors*



OTHER BLOCKS ASSOCIATED WITH A TRY BLOCK

- ▶ `else:`

- ▶ Body of this is executed when execution of associated `try` body **completes with no exceptions**

- ▶ `finally:`

- ▶ Body of this is **always executed** after `try`, `else` and `except` clauses, even if they raised another error or executed a `break`, `continue` or `return`

- ▶ Useful for clean-up code that should be run no matter what else happened (e.g. close a file)

- ▶ Nice to know these exist, but we don't really use these in this class



WHAT TO DO WITH EXCEPTIONS?

- ▶ What to do when encounter an error?
- ▶ **Fail silently:**
 - ▶ Substitute default values or just continue
 - ▶ Bad idea! user gets no warning
- ▶ Return an **“error” value**
 - ▶ What value to choose?
 - ▶ Complicates code having to check for a special value
- ▶ Stop execution, **signal error** condition
 - ▶ In Python: **raise an exception**

```
raise ValueError("something is wrong")
```

keyword

name of error
you want to raise

optional, but typically a
string with a message



EXAMPLE with SOMETHING YOU'VE ALREADY SEEN

- ▶ A function that sums digits in a string
- ▶ Execution stopping means a bad result is not propagated

```
def sum_digits(s):  
    """ s is a non-empty string containing digits.  
    Returns sum of all chars that are digits """  
    assert len(s) != 0, "s is empty"  
    total = 0  
    for char in s:  
        try:  
            val = int(char)  
            total += val  
        except:  
            raise ValueError("string contained a character")  
    return total
```



YOU TRY IT!

```
def pairwise_div(Lnum, Ldenom):  
    """ Lnum and Ldenom are non-empty lists of equal lengths  
    containing numbers  
    Returns a new list whose elements are the pairwise  
    division of an element in Lnum by an element in Ldenom.  
    Raise a ValueError if Ldenom contains 0. """  
    # your code here
```

```
# For example:
```

```
L1 = [4,5,6]
```

```
L2 = [1,2,3]
```

```
# print(pairwise_div(L1, L2)) # prints [4.0,2.5,2.0]
```

```
L1 = [4,5,6]
```

```
L2 = [1,0,3]
```

```
# print(pairwise_div(L1, L2)) # raises a ValueError
```





ASSERTIONS

ASSERTIONS: DEFENSIVE PROGRAMMING TOOL

- ▶ Want to be sure that **assumptions** on state of computation are as expected
- ▶ Use an **assert statement** to raise an AssertionError exception if assumptions not met

```
assert <statement that should be true>, "message if  
not true"
```

- ▶ Assertions don't allow a programmer to control response to unexpected conditions
- ▶ An example of good **defensive programming**
 - ▶ Typically used to **check inputs** to functions, but can be used anywhere
 - ▶ Can be used to **check outputs** of a function to avoid propagating bad values
 - ▶ Ensure that **execution halts** whenever an expected condition is not met
 - ▶ Can make it easier to locate a source of a bug



EXAMPLE with SOMETHING YOU'VE ALREADY SEEN

- ▶ A function that sums digits in a **NON-EMPTY string**
- ▶ Execution stopping means a bad result is not propagated

```
def sum_digits(s):  
    """ s is a non-empty string containing digits.  
    Returns sum of all chars that are digits """  
    assert len(s) != 0, "s is empty"  
    total = 0  
    for char in s:  
        try:  
            val = int(char)  
            total += val  
        except:  
            raise ValueError("string contained a character")
```



YOU TRY IT!

```
def pairwise_div(Lnum, Ldenom):  
    """ Lnum and Ldenom are non-empty lists of equal lengths  
    containing numbers  
    Returns a new list whose elements are the pairwise  
    division of an element in Lnum by an element in Ldenom.  
    Raise a ValueError if Ldenom contains 0. """  
    # add an assert line here
```



LONGER EXAMPLE OF EXCEPTIONS and ASSERTIONS

- ▶ Assume we are given **a class list**: each entry is a list of two parts
 - ▶ A list of first and last name for a student
 - ▶ A list of grades on assignments

Two students, each with a name list and a grades list

```
test_grades = [[['peter', 'parker'], [80.0, 70.0, 85.0]],  
               [['bruce', 'wayne'], [100.0, 80.0, 74.0]]]
```

- ▶ Create **a new class list**, with name, grades, and an average added at the end

```
[[['peter', 'parker'], [80.0, 70.0, 85.0], 78.33333],  
 [['bruce', 'wayne'], [100.0, 80.0, 74.0], 84.666667]]]
```



EXAMPLE CODE

```
[[['peter', 'parker'], [80.0, 70.0, 85.0]],  
 [['bruce', 'wayne'], [100.0, 80.0, 74.0]]]
```

```
def get_stats(class_list):  
    new_stats = []  
    for stu in class_list:  
        new_stats.append([stu[0], stu[1], avg(stu[1])])  
    return new_stats
```

```
def avg(grades):  
    return sum(grades)/len(grades)
```

We will look at variations
of the avg function



ERROR IF NO GRADE FOR A STUDENT

- ▶ If one or more students **don't have any grades**, get an error

```
test_grades = [[['peter', 'parker'], [10.0, 55.0, 85.0]],  
               [['bruce', 'wayne'], [10.0, 80.0, 75.0]],  
               [['captain', 'america'], [80.0, 10.0, 96.0]],  
               [['deadpool'], []]]
```

- ▶ Get `ZeroDivisionError`: float division by zero because try to

```
return sum(grades) / len(grades)
```

length is 0



OPTION 1: FLAG THE ERROR BY PRINTING A MESSAGE

- ▶ Decide to **notify** that something went wrong with a msg

```
def avg(grades):  
    try:  
        return sum(grades)/len(grades)  
    except ZeroDivisionError:  
        print('warning: no grades data')
```

- ▶ Running on same test data gives

```
warning: no grades data
```

```
[[['peter', 'parker'], [10.0, 55.0, 85.0], 50.0],  
 [['bruce', 'wayne'], [10.0, 80.0, 75.0], 55.0],  
 [['captain', 'america'], [80.0, 10.0, 96.0], 62.0],  
 [['deadpool'], [], None]]
```

flagged the error

*because avg did
not return anything
in the except*



OPTION 2: CHANGE THE POLICY

- ▶ Decide that a student with no grades gets a **zero**

```
def avg(grades):  
    try:  
        return sum(grades)/len(grades)  
    except ZeroDivisionError:  
        print('warning: no grades data')  
        return 0.0
```

- ▶ Running on same test data gives

```
warning: no grades data
```

```
[[['peter', 'parker'], [10.0, 55.0, 85.0], 50.0],  
 [['bruce', 'wayne'], [10.0, 80.0, 75.0], 55.0],  
 [['captain', 'america'], [80.0, 10.0, 96.0], 62]  
 [['deadpool'], [], 0.0]]
```

still flag the error

now avg returns 0



OPTION 3: HALT EXECUTION IF ASSERT IS NOT MET

- ▶ Raises an `AssertionError` if it is given an empty list for grades, prints out string message; stops execution
- ▶ Otherwise runs as normal

*function ends
immediately if
assertion not met*

```
def avg(grades):
```

```
    assert len(grades) != 0, 'no grades data'
```

```
    return sum(grades)/len(grades)
```



ASSERTIONS vs. EXCEPTIONS

- ▶ Goal is to **spot problems as soon as introduced** and make clear where they happened
- ▶ Exceptions provide a way of **handling unexpected conditions**
 - ▶ Use when you don't need to halt program execution
 - ▶ Ex: if users supplies bad data input
- ▶ Use **assertions**:
 - ▶ Enforce conditions on a “contract”
 - ▶ As a **supplement** to testing





DICTIONARIES

HOW TO STORE STUDENT INFO

- ▶ Suppose we want to store and use grade information for a set of students
- ▶ Could store using separate lists for each kind of information

```
names = ['Ana', 'John', 'Matt', 'Katy']  
grades = ['A+', 'B', 'A', 'A']  
microquizzes = ...  
psets = ...
```

- ▶ Info stored across lists at same index, each index refers to information for a different person
- ▶ Indirectly access information by finding location in lists corresponding to a person, then extract



HOW TO ACCESS STUDENT INFO

```
def get_grade(student, name_list, grade_list):
```

```
    i = name_list.index(student)
```

```
    grade = grade_list[i]
```

```
    return (student, grade)
```

*find location in
list for person*

*Use location index
to access other info*

- ▶ **Messy** if have a lot of different info of which to keep track, e.g., a separate list for microquiz scores, for pset scores, etc.
- ▶ Must maintain **many lists** and pass them as arguments
- ▶ Must **always index** using integers
- ▶ Must remember to change multiple lists, when adding or updating information



HOW TO STORE AND ACCESS STUDENT INFO

- ▶ Alternative might be to use a list of lists

```
eric = ['eric', ['ps', [8, 4, 5]], ['mq', [6, 7]]]  
ana = ['ana', ['ps', [10, 10, 10]], ['mq', [9, 10]]]  
john = ['john', ['ps', [7, 6, 5]], ['mq', [8, 5]]]  
  
grades = [eric, ana, john]
```

- ▶ Then could access by searching lists, but code is still messy

```
def get_grades(who, what, data):  
    for stud in data:  
        if stud[0] == who:  
            for info in stud[1:]:  
                if info[0] == what:  
                    return who, info
```

```
print(get_grades('eric', 'mq', grades))  
print(get_grades('ana', 'ps', grades))
```

*But idea of associating data
with names is worth exploring*



A BETTER AND CLEANER WAY – A DICTIONARY

- ▶ Nice to use **one data structure**, no separate lists
- ▶ Nice to **index item of interest directly**
- ▶ A Python **dictionary** has entries that map a key:value

Alist

0	Elem 1
1	Elem 2
2	Elem 3
3	Elem 4
...	...

index

element

Adictionary

Key 1	Val 1
Key 2	Val 2
Key 3	Val 3
Key 4	Val 4
...	...

custom index

element



A PYTHON DICTIONARY

- ▶ Store **pairs of data** as an **entry**
 - ▶ key (any immutable object)
 - ▶ str, int, float, bool, tuple, etc
 - ▶ value (any data object)
 - ▶ Any above plus lists and other dicts!

'Ana'	'B'
'Matt'	'A'
'John'	'B'
'Katy'	'A'

empty dictionary

```
my_dict = {}
```

colon maps key:value

```
d = {4:16}
```

custom index by label

element

```
grades = {'Ana':'B', 'Matt':'A', 'John':'B', 'Katy':'A'}
```

↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑

key1 val1 key2 val2 key3 val3 key4 val4

DICTIONARY LOOKUP

- ▶ Similar to indexing into a list
- ▶ **Looks up** the **key**
- ▶ **Returns** the **value** associated with the key
 - ▶ If key isn't found, get an error
- ▶ There is **no simple expression to get a key back given some value!**

'Ana'	'B'
'Matt'	'A'
'John'	'B'
'Katy'	'A'

Key 'John'

Value associated with key 'John'

```
grades = {'Ana': 'B', 'Matt': 'A', 'John': 'B', 'Katy': 'A'}
```

```
grades['John']
```

→ evaluates to 'B'

```
grades['Grace']
```

→ gives a `KeyError`



YOU TRY IT!

- ▶ Write a function according to this spec

```
def find_grades(grades, students):  
    """ grades is a dict mapping student names (str) to grades (str)  
    students is a list of student names  
    Returns a list containing the grades for students (in same order)  
    """  
  
    # for example  
    d = {'Ana':'B', 'Matt':'C', 'John':'B', 'Katy':'A'}  
    print(find_grades(d, ['Matt', 'Katy'])) # returns ['C', 'A']
```



DICTIONARY OPERATIONS

```
grades = {'Ana':'B', 'Matt':'A', 'John':'B', 'Katy':'A'}
```

► **Add** an entry

```
grades['Grace'] = 'A'
```

'Ana'	'B'
'Matt'	'A'
'John'	'B'
'Katy'	'A'
'Grace'	'A'



DICTIONARY OPERATIONS

```
grades = {'Ana':'B', 'Matt':'A', 'John':'B', 'Katy':'A'}
```

▶ **Add** an entry

```
grades['Grace'] = 'A'
```

▶ **Change** entry

```
grades['Grace'] = 'C'
```

▶ **Delete** entry

```
del(grades['Ana'])
```

'Matt'	'A'
'John'	'B'
'Katy'	'A'
'Grace'	'A'



DICTIONARY OPERATIONS

```
grades = {'Ana':'B', 'Matt':'A', 'John':'B', 'Katy':'A'}
```

► **Test** if key in dictionary

'John' in grades	→ returns True
'Daniel' in grades	→ returns False
'B' in grades	→ returns False

The in keyword only checks keys, not values

'Ana'	'B'
'Matt'	'A'
'John'	'B'
'Katy'	'A'



YOU TRY IT!

► Write a function according to these specs

```
def find_in_L(Ld, k):  
    """ Ld is a list of dicts  
    k is an int  
    Returns True if k is a key in any dicts of Ld and False otherwise """  
  
    # for example  
    d1 = {1:2, 3:4, 5:6}  
    d2 = {2:4, 4:6}  
    d3 = {1:1, 3:9, 4:16, 5:25}  
  
    print(find_in_L([d1, d2, d3], 2)) # returns True  
    print(find_in_L([d1, d2, d3], 25)) # returns False
```



DICTIONARY OPERATIONS

'Ana'	'B'
'Matt'	'A'
'John'	'B'
'Katy'	'A'

- ▶ Can iterate over dictionaries but assume there is no guaranteed order

```
grades = {'Ana':'B', 'Matt':'A', 'John':'B', 'Katy':'A'}
```

- ▶ Get an **iterable that acts like a tuple of all keys**

```
grades.keys() → returns dict_keys(['Ana', 'Matt', 'John', 'Katy'])
```

```
list(grades.keys()) → returns ['Ana', 'Matt', 'John', 'Katy']
```

- ▶ Get an **iterable that acts like a tuple of all dict values**

```
grades.values() → returns dict_values(['B', 'A', 'B', 'A'])
```

```
list(grades.values()) → returns ['B', 'A', 'B', 'A']
```



DICTIONARY OPERATIONS

most useful way to iterate over dict entries (both keys and vals!)

'Ana'	'B'
'Matt'	'A'
'John'	'B'
'Katy'	'A'

- ▶ Can iterate over dictionaries but assume there is no guaranteed order

```
grades = {'Ana':'B', 'Matt':'A', 'John':'B', 'Katy':'A'}
```

- ▶ Get an **iterable that acts like a tuple of all items**

```
grades.items()
```

→ returns `dict_items([('Ana', 'B'), ('Matt', 'A'), ('John', 'B'), ('Katy', 'A')])`

```
list(grades.items())
```

→ returns `[('Ana', 'B'), ('Matt', 'A'), ('John', 'B'), ('Katy', 'A')]`

- ▶ Typical use is to **iterate over key,value tuple**

```
for k,v in grades.items():  
    print(f"key {k} has value {v}")
```

key Ana has value B
key Matt has value A
key John has value B
key Katy has value A



YOU TRY IT!

- ▶ Write a function that meets this spec

```
def count_matches(d):  
    """ d is a dict  
    Returns how many entries in d have the key equal to its value """  
  
    # for example  
    d = {1:2, 3:4, 5:6}  
    print(count_matches(d)) # prints 0  
    d = {1:2, 'a':'a', 5:5}  
    print(count_matches(d)) # prints 2
```



DICTIONARY KEYS & VALUES

- ▶ Dictionaries are **mutable** objects (aliasing/cloning rules apply)
 - ▶ Use = sign to make an alias
 - ▶ Use `d.copy()` to make a copy
- ▶ **Assume there is no order** to keys or values!
- ▶ Dict values
 - ▶ Any type (**immutable and mutable**)
 - ▶ Dictionary values can be lists, even other dictionaries!
 - ▶ Can be **duplicates**
- ▶ Keys
 - ▶ Must be **unique**
 - ▶ **Immutable** type (`int`, `float`, `string`, `tuple`, `bool`)
 - ▶ Actually need an object that is **hashable**, but think of as immutable as all immutable types are hashable
 - ▶ Be careful using `float` type as a key



WHY IMMUTABLE/HASHABLE KEYS?

- ▶ A dictionary is stored in memory in a special way
- ▶ Next slides show an example
- ▶ Step 1: A **function is run on the dict key**
 - ▶ The function **maps any object to an int**
E.g. map “a” to 1, “b” to 2, etc
 - ▶ The int corresponds to a position in a block of memory addresses
- ▶ Step 2: At that memory address, **store the dict value**
- ▶ To do a **lookup** using a key, **run the same function**
 - ▶ If the object is immutable/hashable then you get the same int back
 - ▶ If the object is changed then the function gives back a different int!



Hash function

- ▶ Sum the letters
- ▶ Take mod 16 (to fit in a memory block with 16 entries)

$1 + 14 + 1 = 16$
 $16 \% 16 = 0$

Ana	C
-----	---

$5 + 18 + 9 + 3 = 35$
 $35 \% 16 = 3$

Eric	A
------	---

$10 + 15 + 8 + 14 = 47$
 $47 \% 16 = 15$

John	B
------	---

$11 + 1 + 20 + 5 = 37$
 $37 \% 16 = 5$

[K,a,t,e]	B
-----------	---

Memory block (like a list)

0	Ana: C
1	
2	
3	Eric: A
4	
5	[K,a,t,e]: B
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	John: B

Hash function

- ▶ Sum the letters
- ▶ Take mod 16 (to fit in a memory block with 16 entries)

Kate changes her name to Cate. Same person, different name. Look up her grade?

$$3 + 1 + 20 + 5 = 29$$
$$29 \% 16 = 13$$

[C,a,t,e]	B
-----------	---

Not actually allowed.
Key must be immutable!

Memory block (like a list)

0	Ana: C
1	
2	
3	Eric: A
4	
5	[K,a,t,e]: B
6	
7	
8	
9	
10	
11	
12	
13	← ??? Not here!
14	
15	John: B

A PYTHON DICTIONARY for STUDENT GRADES

- ▶ Separate students are separate dict entries

Key 1	Val 1
Key 2	Val 2

```
grades = {'Ana': {'mq': [5, 4, 4], 'ps': [10, 9, 9], 'fin': 'B'},  
          'Bob': {'mq': [6, 7, 8], 'ps': [8, 9, 10], 'fin': 'A'}}
```



A PYTHON DICTIONARY for STUDENT GRADES

- ▶ grades maps str → dict

str		dict
Key 1	'Ana'	
	'mq'	[5, 4, 4]
	'ps'	[10, 9, 9]
Key 2	'fin'	'B'
	'Bob'	
	'mq'	[6, 7, 8]
	'ps'	[8, 9, 10]
	'fin'	'A'

```
grades = {'Ana': {'mq': [5, 4, 4], 'ps': [10, 9, 9], 'fin': 'B'},  
         'Bob': {'mq': [6, 7, 8], 'ps': [8, 9, 10], 'fin': 'A'}}
```



A PYTHON DICTIONARY for STUDENT GRADES

- ▶ The values of grades are dicts
- ▶ Each value maps a
 - ▶ str → list
 - ▶ str → str

Key 1	'Ana'	'mq'	[5, 4, 4]
		'ps'	[10, 9, 9]
		'fin'	'B'
Key 2	'Bob'	'mq'	[6, 7, 8]
		'ps'	[8, 9, 10]
		'fin'	'A'

```
grades = { 'Ana': { 'mq': [5, 4, 4], 'ps': [10, 9, 9], 'fin': 'B' },  
           'Bob': { 'mq': [6, 7, 8], 'ps': [8, 9, 10], 'fin': 'A' } }
```



A PYTHON DICTIONARY for STUDENT GRADES

- ▶ The values of grades are dicts
- ▶ Each value maps a
 - ▶ str → list
 - ▶ str → str

Key 1	'Ana'	'mq'	[5, 4, 4]
		'ps'	[10, 9, 9]
		'fin'	'B'
Key 2	'Bob'	'mq'	[6, 7, 8]
		'ps'	[8, 9, 10]
		'fin'	'A'

```
grades = { 'Ana': { 'mq': [5, 4, 4], 'ps': [10, 9, 9], 'fin': 'B' },  
           'Bob': { 'mq': [6, 7, 8], 'ps': [8, 9, 10], 'fin': 'A' } }
```

```
grades['Ana']['mq'][0] returns 5
```



YOU TRY IT!

```
my_d ={'Ana':{'mq':[10], 'ps':[10,10]},  
      'Bob':{'ps':[7,8], 'mq':[8]},  
      'Eric':{'mq':[3], 'ps':[0]} }
```

```
def get_average(data, what):  
    all_data = []  
    for stud in data.keys():  
        INSERT LINE HERE  
    return sum(all_data)/len(all_data)
```

Given the dict `my_d`, and the outline of a function to compute an average, which line should be inserted where indicated so that `get_average(my_d, 'mq')` computes average for all 'mq' entries? i.e. find average of all mq scores for all students.

- A) `all_data = all_data + data[stud][what]`
- B) `all_data.append(data[stud][what])`
- C) `all_data = all_data + data[stud[what]]`
- D) `all_data.append(data[stud[what]])`



list

vs

dict

-
- ▶ **Ordered** sequence of elements
 - ▶ Look up elements by an integer index
 - ▶ Indices have an **order**
 - ▶ Index is an **integer**
 - ▶ Value can be any type

- ▶ **Matches** “keys” to “values”
- ▶ Look up one item by another item
- ▶ **No order** is guaranteed
- ▶ Key can be any **immutable** type
- ▶ Value can be any type



EXAMPLE: FIND MOST COMMON WORDS IN A SONG'S LYRICS

- 1) Create a **frequency dictionary** mapping `str:int`
- 2) Find **word that occurs most often** and how many times
 - ▶ Use a list, in case more than one word with same number
 - ▶ Return a tuple `(list, int)` for `(words_list, highest_freq)`
- 3) Find the **words that occur greater than X times**
 - ▶ Let user choose X, so allow as parameter
 - ▶ Return a list of tuples, each tuple is a `(list, int)` containing the list of words ordered by their frequency
 - ▶ IDEA: From song dictionary, find most frequent word. Delete most common word. Repeat. It works because you are mutating the song dictionary.



CREATING A DICTIONARY

```
song = "RAH RAH AH AH AH ROM MAH RO MAH MAH"
```

```
def generate_word_dict(song):
```

```
    song_words = song.lower()
```

```
    words_list = song_words.split()
```

```
    word_dict = {}
```

```
    for w in words_list:
```

```
        if w in word_dict:
```

```
            word_dict[w] += 1
```

```
        else:
```

```
            word_dict[w] = 1
```

```
    return word_dict
```

Return is a dict mapping
str:int

Convert all
chars to
lower case

Convert string to list of words;
divides based on spaces

Can iterate over list of
words in song

If word in dict (as a key),
increase # times you've
seen it, update entry

If word not in dict, first time
seeing word, create entry



USING THE DICTIONARY

```
word_dict = {'rah':2, 'ah':3, 'rom':1, 'mah':3, 'ro':1}
```

```
def find_frequent_word(word_dict):
```

```
    words = []
```

```
    highest = max(word_dict.values())
```

```
    for k,v in word_dict.items():
```

```
        if v == highest:
```

```
            words.append(k)
```

```
    return (words, highest)
```

Return is a tuple of (['ah', 'mah'], 3)

Highest frequency in
dict's values

Loop to see which word
has the highest freq

Append to list of all words
that have that highest freq



FIND WORDS WITH FREQUENCY GREATER THAN $x=1$

- ▶ Repeat the next few steps as long as the highest frequency is greater than x

```
word_dict = {'rah':2, 'ah':3, 'rom':1, 'mah':3, 'ro':1}
```



FIND WORDS WITH FREQUENCY GREATER THAN $x=1$

- ▶ Use function `find_frequent_word` to get words with the highest frequency

```
word_dict = {'rah':2, 'ah':3, 'rom':1, 'mah':3, 'ro':1}
```



FIND WORDS WITH FREQUENCY GREATER THAN $x=1$

- ▶ Remove the entries corresponding to these words from dictionary by mutation

```
word_dict = {'rah':2, 'ah':3, 'rom':1, 'mah':3, 'ro':1}
```



FIND WORDS WITH FREQUENCY GREATER THAN $x=1$

- ▶ Remove the entries corresponding to these words from dictionary by mutation

```
word_dict = {'rah':2, 'rom':1, 'ro':1}
```

- ▶ Save them in the result

```
freq_list = [['ah', 'mah'], 3]
```



FIND WORDS WITH FREQUENCY GREATER THAN $x=1$

- ▶ Use function `find_frequent_word` to get words with the highest frequency

```
word_dict = {'rah':2, 'rom':1, 'ro':1}
```

- ▶ The result so far...

```
freq_list = [['ah', 'mah'], 3]
```



FIND WORDS WITH FREQUENCY GREATER THAN $x=1$

- ▶ Remove the entries corresponding to these words from dictionary by mutation

```
word_dict = { 'rom':1, 'ro':1 }
```

- ▶ Add them to the result so far

```
freq_list = ([('ah', 'mah'], 3), ([rah'], 2))
```



FIND WORDS WITH FREQUENCY GREATER THAN $x=1$

- ▶ Use function `find_frequent_word` to get words with the highest frequency
- ▶ The highest frequency is now not greater than $x=1$, so stop

```
word_dict = {'rom':1, 'ro':1}
```

- ▶ The final result

```
freq_list = ([['ah', 'mah'], 3), [['rah'], 2)]
```



LEVERAGING DICT PROPERTIES

```
word_dict = {'rah':2, 'ah':3, 'rom':1, 'mah':3, 'ro':1}
```

```
def occurs Often(word_dict, x):
```

```
    freq_list = []
```

```
    word_freq_tuple = find_frequent_word(word_dict)
```

*Gives us a word tuple
like ('ah', 'mah'), 3)*

```
    while word_freq_tuple[1] > x:
```

*Stay in loop while we still have
frequencies higher than x*

```
        freq_list.append(word_freq_tuple)
```

```
        for word in word_freq_tuple[0]:
```

```
            del(word_dict[word])
```

*Add those words to result
Mutate dict to remove ALL
those words; on next loop, will
find next most common words*

```
        word_freq_tuple = find_frequent_word(word_dict)
```

```
    return freq_list
```



SUMMARY

- ▶ Dictionaries have entries that **map a key to a value**
- ▶ **Keys are immutable/hashable and unique** objects
- ▶ **Values** can be **any object**
- ▶ Dictionaries can make code efficient
 - ▶ Implementation-wise
 - ▶ Runtime-wise

